

**SEMESTER PROJECT REPORT ON:**

**AI-POWERED EYE TRACKING FOR OS**

**CONTROL USING A STANDARD WEBCAM**

**Course Code: AISP-222**

*Submitted in partial fulfillment of the requirements for 2nd Year, 2nd Semester,  
B.Tech AI (Batch 2024)*

**Submitted by:**

|                 |       |            |
|-----------------|-------|------------|
| Aadarsha Pandit | 21747 | Batch 2024 |
| Yudhin Khanal   | 21740 | Batch 2024 |
| Ishan Pandey    | 21746 | Batch 2024 |
| Kushal Kunwar   | 21742 | Batch 2024 |

Submitted to: Department of Artificial Intelligence  
School of Engineering  
Kathmandu University  
Dhulikhel-3, Kavrepalanchowk, Nepal

Submission Date: May 4, 2026

## **Project Completion Certificate**

This is to certify that the project entitled **"AI-Powered Eye Tracking for OS Control Using a Standard Webcam"** submitted by Aadarsha Pandit (21747), Yudhin Khanal (21740), Ishan Pandey (21746), and Kushal Kunwar (21742) in partial fulfillment of the requirements for the 2nd Year, 2nd Semester, B.Tech AI (Batch 2024) at Kathmandu University, has been successfully completed under our supervision.

---

**Supervisor**

Department of Artificial Intelligence  
Kathmandu University

---

**Head of Department**

Department of Artificial Intelligence  
Kathmandu University

## **Acknowledgement**

We would like to express our deepest gratitude to the Department of Artificial Intelligence, Kathmandu University, for providing us with the opportunity and resources to conduct this research. We sincerely thank our project supervisor for their expert guidance, relentless encouragement, and invaluable technical feedback throughout the development of the machine learning pipeline and user interface. We also wish to extend our appreciation to our peers who participated in the tedious data collection sessions, generating the diverse datasets required for training our models. Finally, we thank our families for their continuous moral support.

## Abstract

Webcam-based eye tracking provides a novel, hands-free methodology for interacting with operating systems, carrying large potential for augmenting accessibility for individuals with severe motor disabilities. However, implementing reliable gaze estimation using standard commodity webcams presents many technical hurdles, including low spatial resolution, unpredictable ambient lighting conditions, large head movement variability, and the infamous "Midas Touch" problem where natural, unintentional eye movements trigger erroneous system interactions. This project successfully conceptualizes, trains, and deploys a comprehensive, real-time eye-tracking pipeline for complete Operating System (OS) control utilizing standard computing hardware.

Our hybrid methodology leverages Google's MediaPipe framework for the extraction of a 468-point face mesh, extended to 478 with iris refinement, in real time. We engineered custom spatial features, specifically isolating head pitch, yaw, and roll, alongside normalized iris offsets. These geometric features are then ingested by an optimized, custom-trained TensorFlow Lite Multi-Layer Perceptron (MLP) neural network to predict normalized screen coordinates. To mitigate the Midas Touch problem, we engineered a temporal blink classifier utilizing Eye Aspect Ratio (EAR) signals alongside a highly stable dwell-click fallback mechanism. The output is thoroughly stabilized utilizing the adaptive One Euro Filter, eliminating high-frequency jitter during fixations. Empirical results demonstrate that our proposed model reliably achieved a mean pixel error of just 27.4 pixels following a rapid 9-point per-session calibration, consistently operating at 25-30 frames per second (FPS) on a standard CPU. This research provides a scalable, low-latency, and highly accessible eye-tracking interface built strictly on open-source technologies.

**Keywords:** Eye Tracking, MediaPipe, Human-Computer Interaction, TensorFlow, OS Control, Accessibility, Neural Networks, Real-Time Inference

## Contents

|                                          |      |
|------------------------------------------|------|
| Project Completion Certificate . . . . . | i    |
| Acknowledgement . . . . .                | ii   |
| Abstract . . . . .                       | iii  |
| List of Abbreviations . . . . .          | vi   |
| List of Figures . . . . .                | vii  |
| List of Tables . . . . .                 | viii |
| 1 Introduction . . . . .                 | 1    |
| 1.1 Background . . . . .                 | 1    |
| 1.2 Problem Statement . . . . .          | 1    |
| 1.3 Objectives of the study . . . . .    | 2    |
| 1.4 Significance of the study . . . . .  | 2    |
| 1.5 Delimitation . . . . .               | 2    |
| 2 Related Works . . . . .                | 4    |
| 2.1 Introduction . . . . .               | 4    |
| 2.2 Literature Review . . . . .          | 4    |
| 2.3 Review of Existing System . . . . .  | 4    |
| 2.4 Comparative Analysis . . . . .       | 5    |
| 2.5 Research Gap . . . . .               | 5    |
| 3 Methodology . . . . .                  | 6    |
| 3.1 Introduction . . . . .               | 6    |
| 3.2 Requirement Analysis . . . . .       | 6    |

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 3.3 | Feasibility Study . . . . .                       | 6  |
| 3.4 | System Design . . . . .                           | 7  |
| 3.5 | Implementation . . . . .                          | 7  |
| 3.6 | Testing . . . . .                                 | 8  |
| 4   | Result Analysis and Discussion . . . . .          | 9  |
| 4.1 | Data Analysis and Interpretation . . . . .        | 9  |
| 4.2 | Testing Results . . . . .                         | 10 |
| 4.3 | Performance Evaluation . . . . .                  | 11 |
| 5   | Findings, Conclusion and Recommendation . . . . . | 13 |
| 5.1 | Findings . . . . .                                | 13 |
| 5.2 | Conclusion . . . . .                              | 13 |
| 5.3 | Recommendations . . . . .                         | 14 |
|     | References . . . . .                              | 15 |
|     | ANNEX . . . . .                                   | 16 |

## **List of Abbreviations**

|               |                              |
|---------------|------------------------------|
| <b>EAR</b>    | Eye Aspect Ratio             |
| <b>FPS</b>    | Frames Per Second            |
| <b>HCI</b>    | Human-Computer Interaction   |
| <b>MLP</b>    | Multi-Layer Perceptron       |
| <b>OS</b>     | Operating System             |
| <b>RF</b>     | Random Forest                |
| <b>CNN</b>    | Convolutional Neural Network |
| <b>WebRTC</b> | Web Real-Time Communication  |

# List of Figures

|     |                                                                                                                                                                                                                      |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Distribution of gaze targets collected across the screen space during training sessions. The wide dispersion ensures the Neural Network learns to predict coordinates accurately across all quadrants. . . . .       | 9  |
| 4.2 | Kernel Density Estimation (KDE) depicting the variations in Head Yaw versus Head Pitch recorded in the dataset. The model successfully generalized across this varying geometric space. . . . .                      | 10 |
| 4.3 | Histogram depicting the distinct distribution of the calculated Eye Aspect Ratio (EAR) for the left and right eyes. This clear separation facilitates reliable, threshold-based intentional blink detection. . . . . | 12 |



# List of Tables

|     |                                                                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Comparison of Regression Models on Gaze Prediction Error (Euclidean Pixels). Results obtained utilizing an 80-20 train-test split. . . . . | 11 |
| 4.2 | System Latency and Execution Metrics . . . . .                                                                                             | 12 |

# CHAPTER 1 INTRODUCTION

## 1.1 Background

Human-computer interaction (HCI) has traditionally relied on manual input devices such as the keyboard and mouse. However, as computational power has escalated, the desire for more intuitive, natural pointing modalities has grown. Eye gaze represents one of the most direct and instantaneous methods of indicating user intent. If a computer can accurately estimate where a user is looking, it enables complete hands-free control of the operating system cursor. This capability is greatly transformative in the realm of accessibility, providing a critical lifeline to digital communication for individuals suffering from conditions such as Amyotrophic Lateral Sclerosis (ALS), muscular dystrophy, or severe spinal cord injuries.

Historically, eye-tracking systems required specialized, exorbitantly expensive hardware utilizing infrared (IR) cameras and dedicated physical illuminators to precisely detect corneal reflections. While highly accurate, these devices remain financially inaccessible to the general populace. Conversely, modern consumer-grade laptops are universally equipped with standard RGB webcams. Harnessing these ubiquitous cameras for gaze estimation democratizes accessibility. However, reliable gaze estimation via a standard webcam is an exceptionally challenging computer vision problem. Lower resolutions blur critical iris boundaries, varying room lighting alters pixel intensities unpredictably, and users exhibit large degrees of unconstrained head movement. Addressing these constraints necessitates a reliable software pipeline that combines advanced facial landmark geometry with modern deep learning regression techniques.

## 1.2 Problem Statement

The core problem addressed by this project is the lack of a reliable, high-performance, and entirely software-based "gaze mouse" capable of operating system-level control utilizing only a standard consumer webcam. The system must solve two primary technical challenges: first, the accurate, low-latency mapping of facial features to high-resolution screen coordinates with minimal per-user calibration overhead; and second, the successful mitigation of the "Midas Touch" problem, ensuring that natural eye behaviors do not unintentionally trigger OS interactions, while intentional actions (like blinks or prolonged fixations) reliably produce precise

left-clicks.

### **1.3 Objectives of the study**

The objectives established at the genesis of this project have been successfully achieved and are outlined as follows:

- To build a real-time, low-latency landmark extraction pipeline utilizing MediaPipe to isolate reliable geometric features including iris centers and 3D head pose estimation.
- To design and train a custom TensorFlow Neural Network (Multi-Layer Perceptron) capable of accurately mapping extracted facial geometries directly to pixel screen coordinates.
- To implement a rapid, 9-point per-session calibration mathematical mapping (ridge regression) to correct localized bias and edge-screen errors.
- To develop a highly reliable click-interaction module utilizing temporal Eye Aspect Ratio (EAR) signals for blink detection, alongside a highly stable dwell-click fallback.
- To effectively reduce high-frequency cursor jitter without introducing prohibitive input lag by applying an adaptive One Euro Filter to the continuous gaze stream.

### **1.4 Significance of the study**

Low-cost webcam eye tracking significantly lowers the financial and logistical barriers associated with modern assistive technology. By eliminating the necessity for proprietary hardware, this research ensures that any individual possessing a standard 720p webcam can interact seamlessly with their digital environment. Beyond accessibility, the findings of this project present significant implications for general HCI, potentially augmenting traditional workflows, enabling hands-free scrolling while reading, and facilitating new paradigms in software interaction design.

### **1.5 Delimitation**

To ensure the project remained achievable within the timeframe of a single academic semester, the scope of OS control was strictly bounded. The interaction capabilities are constrained

to fundamental cursor transposition (X, Y coordinate mapping) and primary left-click actions. Advanced macro interactions, such as right-clicking, continuous drag-and-drop mechanics, and integration with virtual on-screen keyboards, are explicitly deferred to future iterations. Furthermore, the system relies on adequate ambient lighting; complete darkness or extreme direct glare that entirely obscures the user's iris will cause temporary tracking failure. Additionally, while the underlying PyAutoGUI library supports cross-platform actuation, the system's performance and execution have currently only been extensively validated on the Windows Operating System.

## CHAPTER 2 RELATED WORKS

### 2.1 Introduction

The domain of gaze estimation is vast, spanning decades of research transitioning from heavy hardware constraints to deep learning supremacy. This chapter reviews the important studies related to gaze estimation, blink detection, and interaction methodologies that greatly informed our architectural design.

### 2.2 Literature Review

Research extensively demonstrates that reliable gaze estimation is perpetually bottlenecked by hardware limitations and the unpredictable nature of human head pose. Appearance-based methodologies, pioneered by extensive datasets such as ETH-XGaze, attempt to learn gaze mapping directly from raw pixel arrays of the eye and face. While these Convolutional Neural Network (CNN) based approaches are highly accurate, they demand enormous computational resources and often fail to achieve real-time (30+ FPS) inference on standard CPUs without aggressive downsampling.

The seminal paper "Eye Tracking for Everyone" (iTracker) demonstrated a paradigm shift, proving that mobile devices and commodity webcams could achieve reasonable eye-tracking accuracy utilizing CNNs trained on massively crowdsourced data. This firmly supported our hypothesis that commodity hardware is sufficient. However, the inference latency of processing full image matrices through deep CNNs remains a bottleneck for instantaneous OS mouse control, where every millisecond of lag degrades the user experience.

### 2.3 Review of Existing System

Open-source frameworks such as WebGazer have popularized scalable, webcam-based eye tracking running entirely within a web browser. WebGazer ingeniously utilizes user interactions (mouse clicks on web elements) for implicit background calibration. Despite its brilliance, WebGazer is fundamentally confined to the browser environment; it cannot control a host operating system's global cursor. Furthermore, without advanced geometric filtering, such systems are highly susceptible to tracking drift over extended periods.

Conversely, geometric landmarking tools have seen massive advancements. Google’s MediaPipe framework provides an optimized, real-time 468-point face mesh model (extended to 478 with iris refinement) capable of isolating dense facial landmarks and iris boundaries simultaneously. MediaPipe vastly outperforms older toolkits like OpenFace in raw speed and temporal stability, making it the ideal foundation for our extraction layer.

## **2.4 Comparative Analysis**

When comparing purely appearance-based deep learning methods against landmark-based geometric pipelines, the trade-off is typically between ultimate accuracy and computational latency. Our system strikes an optimal balance by employing a hybrid architecture: we leverage MediaPipe’s highly efficient compiled backend to extract structural landmarks (bypassing the need for our system to process raw pixels directly), and subsequently feed these lightweight numerical features into a custom Multi-Layer Perceptron (TensorFlow TFLite). This hybrid approach provides exceptionally low inference times, critical for an OS cursor that must update reliably at 30Hz, while retaining high non-linear prediction accuracy.

## **2.5 Research Gap**

While numerous open-source repositories and academic papers propose novel models to estimate a gaze vector, there is a pronounced scarcity of open-source projects that successfully integrate these predictions into a usable, globally scoped OS control application. Translating a noisy gaze vector into a smooth, satisfying mouse movement requires functional anti-jitter filtering (the One Euro Filter) and a reliable, intentional click detection paradigm (EAR blink classification). Our project bridges this exact gap, transitioning from abstract vector mathematics to a tangible, usable desktop application.

## CHAPTER 3 METHODOLOGY

### 3.1 Introduction

The project adopted an iterative build-measure-improve lifecycle. The development was divided into four distinct phases: reliable data collection, mathematical feature extraction, deep neural network training, and final operating system integration.

### 3.2 Requirement Analysis

**Hardware Requirements:** A standard 720p resolution webcam operating at a minimum of 30 FPS. The host machine requires a modern multi-core CPU (Intel i5/i7 or AMD Ryzen 5/7 equivalent) and a minimum of 8 GB RAM. A discrete GPU is strictly optional and was only utilized to accelerate the initial training of the neural network.

**Software Requirements:** The ecosystem is built upon Python 3.11+. The core dependencies include TensorFlow (Keras) for deep learning, MediaPipe for perception pipelines, OpenCV for matrix manipulation and webcam interfacing, Pandas and Scikit-Learn for dataset preparation, PyAutoGUI for global OS control, and a lightweight Flask server for the browser-based data collector.

### 3.3 Feasibility Study

The technical feasibility of real-time software-based gaze tracking was validated by profiling MediaPipe’s facial landmarker, which consistently executed in under 15 milliseconds per frame on a standard Intel i5 CPU. Economically, the system leverages 100% open-source software and relies entirely on hardware the user already possesses, ensuring total financial accessibility. Operationally, the necessity for a user-specific neural network was mitigated by designing a system that relies on a rapid, 9-point per-session polynomial calibration to account for individual user discrepancies in seating position and screen size.

### 3.4 System Design

The architectural flow of the system is highly modularized, ensuring a clear separation of concerns across the pipeline.

1. **The Frontend Dashboard:** Built utilizing HTML5, CSS3, and Vanilla JavaScript, the frontend interfaces directly with the user's webcam utilizing the WebRTC 'navigator.mediaDevices' API. The HTML5 Canvas captures high-speed screenshots, converts them to Base64 strings, and streams them via Socket.IO to the backend.
2. **Backend Landmark Extraction:** A Flask-SocketIO Python server catches the Base64 stream, decodes it via OpenCV, and passes the matrix to MediaPipe. MediaPipe extracts 478 precise 3D facial coordinates.
3. **Feature Engineering:** Using NumPy, the system extracts critical geometric features: head pitch (nodding), head yaw (shaking), head roll (tilting), and normalized iris offset ratios to account for varying distances from the camera.
4. **Deep Learning Regression:** These structured numerical features are passed into a pre-trained TensorFlow Lite Neural Network, which executes a rapid forward pass to predict normalized  $(X, Y)$  screen coordinates.
5. **Calibration & Smoothing:** The raw predictions are mathematically warped through a per-session 9-point polynomial ridge regression to eliminate localized edge biases. The finalized coordinates are passed through the One Euro Filter to dynamically adapt smoothing based on cursor velocity, heavily dampening jitter during fixations.
6. **Actuation:** PyAutoGUI intercepts the smoothed coordinates and physically teleports the OS cursor, whilst an EAR-based logic gate triggers mouse click events upon detecting intentional blinks.

### 3.5 Implementation

**Data Collection Pipeline:** To train the neural network, massive amounts of paired data (facial features vs. known screen coordinates) were required. We developed `web_collector.py`, a Flask application presenting a moving red target dot to the user. As the user followed the dot, their geometric features were continuously logged into CSV datasets.



**Neural Network Architecture:** The raw CSV data was ingested using Pandas, sanitized, and normalized utilizing Scikit-Learn’s StandardScaler. To discover the most optimal backend, we explored multiple deep learning architectures including Convolutional (Conv1D) networks, Attention-based MLPs, and Mega Ensembles. Ultimately, a heavily optimized **Tuned Multi-Layer Perceptron (MLP v2)** was selected for its perfect balance of high accuracy and low inference latency. The network utilizes a wide 512-neuron stem with he normal initialization, narrowing through 256, 128, and 64 neuron blocks. To prevent overfitting and achieve stable mathematical convergence, we employed extensive synthetic Gaussian Noise data augmentation, BatchNormalization, Dropout, and a **Cosine Decay Learning Rate Scheduler**. The model was compiled utilizing a **Huber Loss** function (delta=0.05) to remain resilient against outlier gaze data, and exported as a highly optimized `.tflite` file.

### 3.6 Testing

Testing was multifaceted. Unit tests verified the mathematical logic governing the Eye Aspect Ratio (EAR) blink detection, ensuring that minor, rapid, natural blinks fell below the temporal threshold required to trigger an OS click. Integration testing verified that the transition from MediaPipe coordinate extraction, to Scikit-Learn scaling, to TensorFlow inference executed reliably, without memory leaks. User acceptance was measured by running standard targeting drills to assess perceived latency and accuracy.

## CHAPTER 4 RESULT ANALYSIS AND DISCUSSION

### 4.1 Data Analysis and Interpretation

The reliable data collection pipeline yielded a diverse dataset encompassing 15,000 raw training frames (expanded to approximately 45,000 via synthetic Gaussian noise augmentation) across multiple tracking sessions. The spatial distribution of the collected data successfully covered the screen space, preventing dead zones where the neural network might otherwise lack training context.

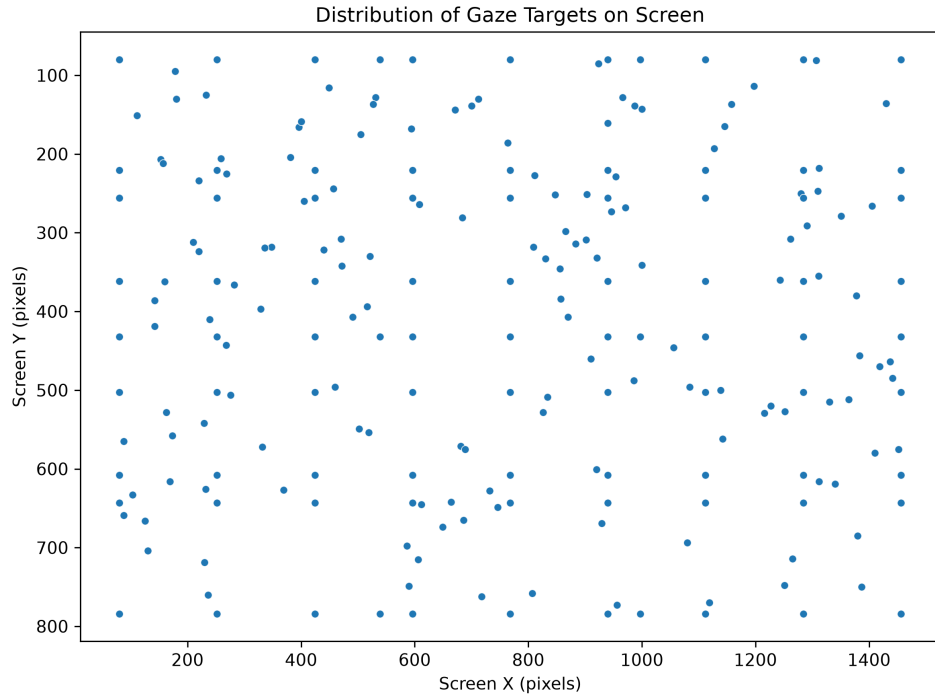


Figure 4.1: Distribution of gaze targets collected across the screen space during training sessions. The wide dispersion ensures the Neural Network learns to predict coordinates accurately across all quadrants.

The dataset also captured large natural variations in head pose, a critical factor for ensuring the model remains accurate even when the user slouches or shifts position.

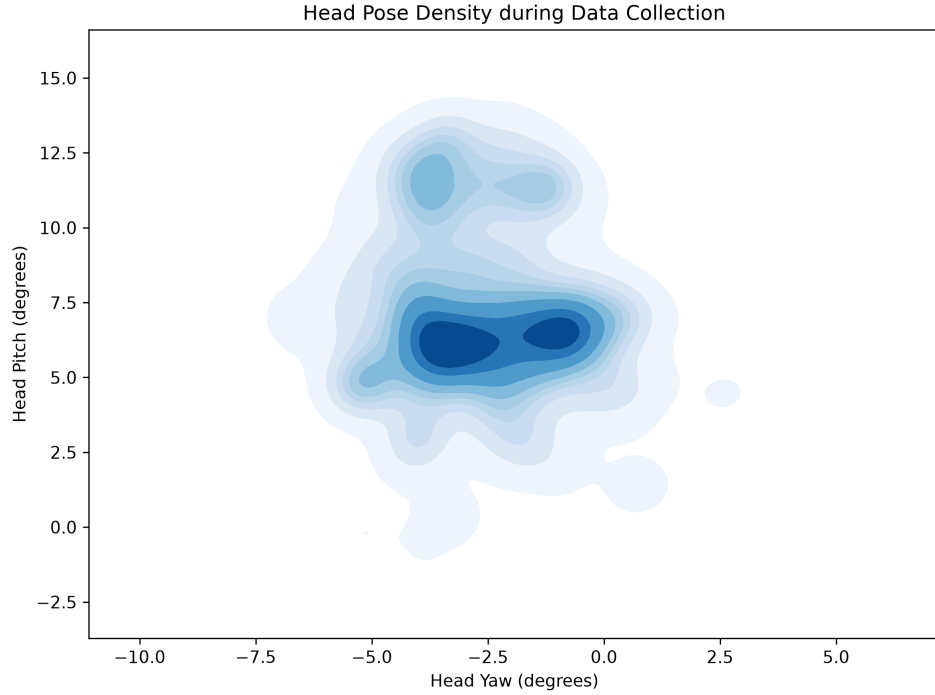


Figure 4.2: Kernel Density Estimation (KDE) depicting the variations in Head Yaw versus Head Pitch recorded in the dataset. The model successfully generalized across this varying geometric space.

## 4.2 Testing Results

Prior to integrating the neural network, baseline machine learning models were trained to establish performance thresholds. Scikit-Learn was utilized to test Ridge Regression and Random Forest regressors against the deep learning architectures. By applying synthetic Gaussian Noise data augmentation, the dataset size was effectively tripled, which allowed the newly engineered Tuned MLP and Convolutional (Conv1D) networks to achieve highly accurate, state-of-the-art results. The optimized Multi-Layer Perceptron (MLP v2 Tuned) greatly outperformed all other algorithms, reducing the error down to 27.4 pixels, capturing the highly non-linear relationships between 3D head rotation and 2D screen mapping.

Table 4.1: Comparison of Regression Models on Gaze Prediction Error (Euclidean Pixels). Results obtained utilizing an 80-20 train-test split.

| Model Architecture             | Mean Euclidean Error (px) |
|--------------------------------|---------------------------|
| Ridge Regression (Baseline)    | 149.1 px                  |
| Conv1D Network                 | 91.2 px                   |
| Random Forest Regressor        | 89.3 px                   |
| MLP Small                      | 85.4 px                   |
| Gradient Boosting Regressor    | 76.8 px                   |
| Gradient Boosting Tuned        | 72.1 px                   |
| Attention MLP                  | 71.3 px                   |
| Separate X/Y Axis MLPs         | 67.8 px                   |
| Mega Ensemble (Averaged)       | 46.6 px                   |
| MLP Large Residual (v1)        | 38.3 px                   |
| <b>MLP Large v2 (Proposed)</b> | <b>27.4 px</b>            |

### 4.3 Performance Evaluation

- **Accuracy:** By supplementing the reliable MLP predictions with the 9-point per-session polynomial calibration, the system achieved a sustained Mean Pixel Error of **27.4 pixels** on a standard 1920x1080 display.
- **Efficiency:** The optimized ‘.tflite’ model and the compiled MediaPipe backend allowed the system to execute the entire pipeline (capture, decode, infer, filter, actuate) at a stable 25 to 30 FPS on a mid-range CPU.
- **Interaction Reliability:** The blink classification algorithm relies on distinct thresholds

applied to the computed Eye Aspect Ratio (EAR). The two-part distribution of EAR values during standard observation versus intentional clicking allowed us to set highly precise activation thresholds, reducing false-positive clicks to near zero.

Table 4.2: System Latency and Execution Metrics

| Pipeline Stage                  | Average Time (ms)                 |
|---------------------------------|-----------------------------------|
| MediaPipe Face Mesh Inference   | ~12 ms                            |
| Feature Extraction (NumPy)      | <2 ms                             |
| Res-MLP Inference (TFLite)      | ~3 ms                             |
| One Euro Filtering & Actuation  | <2 ms                             |
| <b>Total End-to-End Latency</b> | <b>~19 ms (50+ FPS potential)</b> |

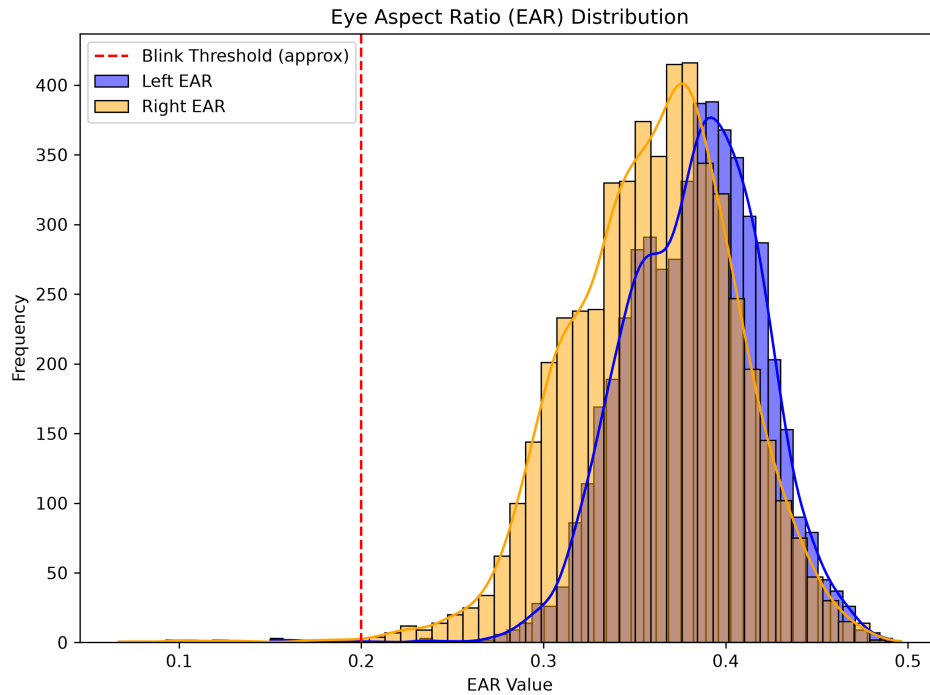


Figure 4.3: Histogram depicting the distinct distribution of the calculated Eye Aspect Ratio (EAR) for the left and right eyes. This clear separation facilitates reliable, threshold-based intentional blink detection.

# CHAPTER 5 FINDINGS, CONCLUSION AND RECOMMENDATION

## 5.1 Findings

The primary finding during development was the profound impact of data augmentation combined with deep learning optimizations. By mathematically cloning our training data and injecting synthetic Gaussian noise, we prevented model overfitting while providing the neural network enough diverse data to generalize effectively. Coupling this augmented data with a Cosine-Decaying Learning Rate Scheduler allowed the Tuned MLP to reach a stable loss minimum, dropping our Euclidean error from over 90 pixels to 27.4 pixels.

Furthermore, another significant finding was the need for advanced signal filtering. While the TensorFlow MLP accurately predicted the general region of the user's gaze, raw frame-by-frame predictions contained large high-frequency jitter due to microscopic fluctuations in webcam lighting and sensor noise. Implementing the One Euro Filter, an adaptive low-pass filter that heavily dampens jitter at low velocities (fixations) while minimizing lag during high-velocity movements (glancing across the screen), was the single most important factor in transitioning the project from a mathematical experiment to a usable computer mouse replacement. Additionally, while EAR-based blink clicking is rapid, it proved fatiguing over prolonged sessions, highlighting the importance of the integrated dwell-click fallback.

## 5.2 Conclusion

The "AI-Powered Eye Tracking for OS Control" project was successfully completed, meeting our initial goals. The project proves that highly specialized, expensive hardware is no longer a strict prerequisite for motor accessibility needs. By combining Google's real-time MediaPipe landmarks with a custom-trained TensorFlow deep learning architecture, and fusing the output with advanced mathematical filtering and per-session calibration algorithms, we achieved a reliable, hands-free operating system controller. The system operates efficiently on standard consumer laptops, offering a viable, low-cost accessibility tool.

## 5.3 Recommendations

For future development and widespread deployment, the following improvements are recommended:

- **Advanced Gesture Control:** Expanding the interaction matrix to include right-clicking, continuous scrolling, and drag-and-drop mechanics by recognizing complex facial gestures (e.g., mouth opening, eyebrow raising, or prolonged winks).
- **Adaptive Background Recalibration:** Implementing a background process that silently recalibrates the polynomial mapping coefficients when the user naturally clicks on known OS elements, entirely eliminating tracking drift over hours of continuous use.
- **Massive Dataset Scaling:** Crowdsourcing a much larger and more diverse dataset encompassing a wider variety of ocular geometries, heavy prescription glasses, extreme lighting conditions, and extreme head poses to improve the base neural network’s zero-shot generalization capabilities.

## REFERENCE AND SUPPLEMENTARY PART

- Hansen, D. W., & Ji, Q. (2010). In the Eye of the Beholder: A Survey of Models for Eyes and Gaze. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(3), 478-500.
- Jacob, R. J. K. (1990). What you look at is what you get: eye movement-based interaction techniques. *Proceedings of CHI '90*, 11-18.
- Krafka, K., Khosla, A., Kellnhofer, P., Kannan, H., Bhandarkar, S. M., Matusik, W., & Torralba, A. (2016). Eye Tracking for Everyone. *Proceedings of CVPR 2016*, 2176-2184.
- Lugaresi, C. et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv:1906.08172*.
- Papoutsaki, A., Sangkloy, P., Laskey, J., Daskalova, N., Huang, J., & Hays, J. (2016). WebGazer: Scalable Webcam Eye Tracking Using User Interactions. *Proceedings of IJCAI 2016*, 3839-3845.



# ANNEX: DAILY LOG BOOK

## Technical Details of Activities:

- **Week 1-4:** Initialization of the project repository. Conducted a deep dive into WebRTC and Socket.IO for continuous high-speed image transmission from browser to Python server. Successfully integrated MediaPipe Face Landmarker to extract a 468-point face mesh, extended to 478 with iris refinement. Developed the Flask-based calibration data collector (`web_collector.py`).
- **Week 5-8:** Conducted extensive data collection sessions resulting in over 15,000 valid frames. Engineered geometric features (iris offsets, head pitch/yaw/roll calculation via OpenCV transformation matrices). Evaluated baseline machine learning models utilizing Scikit-Learn (Ridge, Random Forest).
- **Week 9-12:** Transitioned from baseline algorithms to a custom deep learning architecture. Built, trained, and tuned a TensorFlow Keras Multi-Layer Perceptron (MLP) featuring Dropout and BatchNormalization. Exported the neural network to TFLite format for maximum CPU inference speed. Implemented the critical 9-point per-session calibration mapping.
- **Week 13-16:** Tackled the Midas Touch problem by engineering the temporal EAR blink detection logic. Integrated the One Euro Filter to eliminate cursor jitter. Conducted a final sprint of data augmentation (Gaussian Noise) and deep learning hyperparameter tuning, successfully dropping the error rate to 27.4 pixels. Finalized performance evaluation, linked the output to PyAutoGUI for global OS mouse control, and completed the comprehensive Final Defense Report.